46. ML-Adjacent Pipelines: ETL, Batching, Features

46. ML-Adjacent Pipelines: ETL, Batching, Features

Phase: Phase 7 — AI engineering in Go
Estimated time: 2 days
Slug: `46-ml-pipelines`

Learning objectives

- Build ETL stages as interfaces
- Batch records for throughput
- Validate feature schemas
- Make pipelines restartable

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Pipeline stages

```
type Stage[In, Out any] interface {
    Process(ctx context.Context, in In) (Out, error)
}
```

Compose extract → transform → load. Each stage should be testable offline.

Batching

Micro-batches improve throughput to model APIs and DBs. Flush on size or time (`time.Ticker`).

Features

Document types/units. Version feature schemas. Reject bad rows early with metrics.

Idempotent loaders let you replay safely after failures.

Common mistakes (especially from Python instincts)

- Giant unbounded buffers
- Schema-free JSON forever
- Silent dropping of bad rows without metrics

Exercises

Work in `exercises/46-ml-pipelines/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/46-ml-pipelines/` until you have an honest failing attempt.

Self-check

1. Why batch?
2. What makes a pipeline restartable?
3. Stage interface benefit?

Further reading

- https://go.dev/blog/pipelines